The proof-refinement harness: a system description

Companion document to the main paper. This appendix describes the multi-agent harness that drove the certified upper bound on $\mathcal{K}_G$ from the Braverman et al. baseline ($\mathcal{K}_G \le \pi/(2\ln(1+\sqrt{2})) - 3{\cdot}10^{-32}$) to the present $\mathcal{K}_G \le \pi/(2\ln(1+\sqrt{2})) - 10^{-17}$. The harness is file-based, agentic, and structured around a small number of explicit guarantees about who may write what. We describe the components in the order they fire during a single iteration of the loop.

1 Goal and game

Lemma 5.4 of [1] bounds the second-order Taylor remainder of an implicit-function family at $p = 0$ by $C_r\, p^2$. The constant C_r propagates verbatim into the certified $\mathcal{K}_G$-upper-bound: shrinking C_r shrinks the gap to Krivine.

The harness curates a sequence of *candidates* $V_0, V_1, V_2, \ldots, V_N$, each a self-contained Lean 4 file in `ProofRefinement/Lemma54/Candidates/` that proves Lemma 5.4 with a declared rational ${C_r^{\text{val}}}_K$. The seed V_0 mirrors a paper-faithful production proof; later candidates explore departures. The harness's only goal is

$$V_{N+1} \text{ exists, compiles, audits clean, and has } {C_r^{\text{val}}}_{N+1} < {C_r^{\text{val}}}_N.$$

Strictly smaller score wins. Everything else—formalization style, oracle technology, formula choice—is means.

2 The artifacts (file-based state)

The harness has no in-memory state. Every fact that survives across agent invocations lives in one of these files. The table below is the complete enumeration; nothing else persists.

Path	Purpose
`Candidates/V<K>.lean`	Append-only sequence of Lean candidates. V_0 is the seed.
`Sprint_Contracts/sprint_<NNN>.md`	One contract per planned sprint, written by the PLANNER. Append-only.
`Refinement_Log/log.jsonl`	Append-only JSONL: one line per evaluator run (compile, axiom audit, score, oracle results, timing).
`Refinement_Log/verdict_<NNN>.md`	Markdown verdict file written by the EVALUATOR per sprint.
`Bulletin.md`	Volatile signaling between sprints ($\leq$100 lines, trimmed). The current priority directive lives here.
`Experience.md`	Narrative agent log: what was tried, what worked, what didn't. Written by the GENERATOR.
`Scratch/sprint_<NNN>_<topic>.{py,md,txt}`	Persistent research notebook. Numerical experiments by all roles.
`Proposals/proposal_<NNN>_<slug>.md`	Human-approval queue for non-oracle trust expansions. Status: `pending`/`approved`/`rejected`.
`lemma.toml`	Single source of truth for the sink theorem, the score expression, the seed-axiom allowlist, and the oracle table.
`PATCH_NOTES.md`	Binding-rule document for any rule introduced retroactively (e.g., the $K_{\max}$-truncation regression of 2026-05-03).
`Cache/Backups/<date>_<purpose>/`	Cached canonical numerical outputs (production chunks, subdivision rounds, audit JSONs).
`Numerical_Library/`	Python package of certified-arb oracle implementations, indexed by `lemma.toml`'s `[[oracles]]` table.
`Proof/Lib/Lemma54_Paper*.lean`	Shared infrastructure: Cauchy estimates, identity (75), Taylor-in-p machinery. Editable by the GENERATOR only when the contract names them.

The discipline is exact: every other file is read-only or absent. In particular there is no shared in-memory orchestrator; each agent invocation is a fresh process that reads files, writes files, and exits.

3 The three roles

The harness consists of three agent roles, each defined by a Markdown prompt in `.claude/agents/`. They are spawned sequentially, never concurrently, and never co-resident in the same process.

3.1 The Planner

Inputs. Bulletin, Experience, the last 10 log entries, the last contract, the production trust-boundary status, the agent brief, all proposals (just their `Status` lines), and PATCH_NOTES.

Output. *Exactly one* Sprint Contract file `Sprint_Contracts/sprint_<NNN>.md` per invocation, plus optional Bulletin appends and Scratch/ exploration scripts.

Discipline. The Planner never edits Lean. Its sole deliverable is a contract that prescribes what edits the GENERATOR should make. It must (i) work out the right move via `Scratch/` numerical experiments before drafting (*Scratch-first*), (ii) not repeat the same direction more than three sprints in a row (*no-same-axis-three-times*), and (iii) schedule a wildcard sprint branched from V_0 at least every fourth sprint (*forced trajectory escape*).

Sprint Contract structure. A contract is a Markdown file with a fixed set of fields:

- **Parent candidate.** Which V_K the new candidate branches from (typically V_{N-1}, but explicit non-default parents are permitted and required for wildcard sprints).
- **Target file.** The new candidate path V_{N+1}`.lean` (or, rarely, a research script under `Scratch/`).
- **Direction.** A short tag such as `formula-change`, `contour-change`, `inline-oracle`, `oracle-resolution-tig research`.
- **Tier.** `cheap` (30 min, surgical) or `ambitious` (3 h, substantial new Lean).
- **Goal** (one declarative sentence).
- **Mathematical insight** (5 sentences). What inequality, identity, or proof-method shift is being attempted, with citations to `Proof/Lib/Lemma54_Paper*.lean` or to the paper.
- **Why** (3 sentences with numerical evidence). Cite a `Scratch/sprint_<NNN>_planner_<topic>.py` run that supports the proposed move.
- **Concrete edits expected.** The exhaustive list of files the Generator may write to. Anything else is a write-scope violation (gate G3, below).
- **Acceptance criteria.** 2–5 mechanically checkable `bash` / `jq` / `grep` predicates — each one either an exit code or a `log.jsonl` field comparison.
- **Out of scope.** Explicit no-touch list, beyond the always-forbidden defaults.
- **Time budget** and **Stop conditions for the Generator.** Including the labeled status sentinels `STATUS=READY_FOR_EVALUATION`, `STATUS=BLOCKED`, `STATUS=BLOCKED_NEEDS_HUMAN`.

A typical contract is 2–6 pages, with the bulk of the content in **Mathematical insight** and **Concrete edits**. The contract is the harness's *atomic unit of work*: one contract → one Generator session → one Evaluator session → one verdict → at most one new candidate.

3.2 The Generator

Inputs. The contract path it was given; the parent candidate; the brief; PATCH_NOTES; the specific files the contract names.

Output. The new candidate file (or research script); optional Edits to the files the contract names; an Experience.md append; a labeled terminating `STATUS=` sentinel.

Discipline. The Generator never grades its own work, never edits the contract, never edits older candidates, never edits the Evaluator or runner code, never adds a new (non-oracle) axiom. The write-scope is gated mechanically: only the files in the contract's **Concrete edits expected** list, plus three always-allowed paths (`Experience.md`, `Bulletin.md` (one-line append), `Scratch/`). Everything else is a write-scope violation.

Inline oracle creation. A central feature: if the mathematical move requires a new sup-norm bound on an analytic expression over the oracle rectangle $\Omega(\delta, B)$, the Generator mints the oracle *in the same sprint*, without going through the proposal queue. The recipe is:

1. Write `Numerical_Library/m_<name>.py` computing $\sup_{w \in \Omega(\delta,B)} |\mathrm{expr}(w)|$ via certified-arb interval arithmetic, returning an `OracleResult(bound: Fraction, ball_value: float, ...)`.
2. Append a new `[[oracles]]` block to `lemma.toml` with the axiom short name, the bound-field, and the `oracle_fn` attribute name.
3. Declare a Lean axiom in the candidate of the form $\forall p \in [0,1]$, $\forall w \in \Omega(\delta, B)$, $\|\mathrm{expr}(p, w)\| \leq \mathrm{bound_field_val} \cdot \|D(p, w)\|^k$, which the Generator then consumes in the proof.

This is safe because the runtime arb pipeline re-verifies every oracle on *every* audit (gate G5). A wrong Python implementation is caught structurally by the bound-field-vs-recomputed-bound inequality and content-wise by the human's morning-after diff review. Oracle-shaped axioms cannot be used to "prove" arbitrary statements: their syntactic form constrains them to numerical sup-norm claims.

3.3 The Evaluator

Inputs. The contract path; the candidate file; access to `lake env lean` and `Numerical_Library`.

Output. A Markdown verdict file `Refinement_Log/verdict_<NNN>.md` and a single appended JSON line to `log.jsonl`. Terminates with `STATUS=VERDICT_PASS` or `STATUS=VERDICT_FAIL`.

Discipline. The Evaluator never edits Lean, never argues with a failing gate, never reads the Generator's narrative when deciding pass/fail. Gates are binary.

Gates (in order, terminating on first failure). The seven gates are deliberately cheap. Aggregate runtime ∼30 s.

G1: Compile (focused).
After clearing stale `.olean`, `lake env lean` the new candidate. Pass = exit 0, no `error:` lines.

G2: No `sorry`.
`grep` the candidate. Any match fails.

G3: Write scope.
`git diff --name-only HEAD` must be a subset of (contract-named files) ∪ (always-allowed paths) ∪ (a new V_{N+1} file). Older candidates are sacrosanct.

G4: No new non-oracle axioms.
Compare the new candidate's `##print axioms` set to the parent's. New axioms are permitted only if their `kind = oracle` and they appear in `lemma.toml`'s `[[oracles]]` table. Seed or unregistered axiom additions fail with `NEW_AXIOM_INTRODUCED`.

G5: Per-candidate evaluator.
Run `Evaluator.py --file <cand>`. This recomputes every oracle bound by calling its registered `oracle_fn`, and checks bound_field_val $\geq$ oracle_fn().bound. Pass = exit 0 and `pass = true` in the JSON record.

G7: Rigor (anti-truncation + audit-floor).
Two parts; both must pass. *G7.a* `grep`s the candidate and every oracle source it depends on for the four forbidden symbols (`eval_H_eta_at_acb`, `eval_phi_at_acb`, `eval_H_eta_deriv_at_acb`, `eval_phi_and_deriv_at_acb`). These are the $K_{\max}$-truncated evaluators that admit no rigorous tail bound; any candidate whose oracle path reaches them is invalid by construction. *G7.b* parses the candidate's mandatory audit-block header (PATCH_NOTES Rule R5), looks up the named row in the cached audit JSON, and checks ${C_r^{\text{val}}}_{\text{declared}} \geq {C_r^{\text{patched}}}_{\text{audit}}$. This binary reconciliation between the declared and the truncation-free-recomputed bound is the gate that would have caught the $K_{\max}$ regression had it existed earlier.

G6: Score.
Parse the just-appended `log.jsonl` line. Confirm ${C_r^{\text{val}}}_{N+1} < {C_r^{\text{val}}}_{N}$ (or whatever inequality the contract declared).

The order *G5* → *G7* → *G6* is deliberate: a candidate that has already passed the per-candidate evaluator should still be rejected if its rigor floor is wrong, before its score is even compared.

4 The trust boundary

Every axiom transitively depended on by the sink theorem `lemma_5_4_search` falls into exactly one of three classes, recognized by `Evaluator.py` via `lemma.toml`.

Lean primitives. `propext`, `Classical.choice`, `Quot.sound`. Always permitted.

Seed axioms. A whitelist in `lemma.toml:[seed_axioms.allowed]`. The 12 paper-cited analytic axioms about the implicit-function family F_{paper}^{-1} (`F_inv_paper`, `F_inv_paper_eq`, `F_inv_paper_zero`, `F_inv_paper_R_bound`, `F_inv_paper_analytic`, `F_inv_paper_contdiff`, `F_inv_paper_first_deriv_zero`, `F_inv_paper_mem_S`, `pr_paper`, `pr_paper_pos`, `pr_paper_le_one`, and the structural domain-containment `h_oracle_domain_contains_image`) live here. Each represents a fact the formalization takes on paper-faith.

Oracle axioms. An array of `[[oracles]]` records, each of which has the form

```
[[oracles]]
axiom       = "h_<name>_oracle"
bound_field = "<field>_val"
oracle_fn   = "compute_<name>_truncfree_bound"
```

The Lean axiom must be of the sup-norm shape $\forall p \in [0,1]$, $\forall w \in \Omega(\delta, B)$, $\|\text{expr}(p,w)\| \leq$ <field>_val$\cdot \|D(p,w)\|^k$. At every audit, the Evaluator runs the registered `oracle\_fn` (certified arb computation) and rejects the candidate if the declared `<field>\_val` fails to dominate the recomputed bound.

Anything else (`sorryAx`, an unregistered axiom, an oracle whose short-name is missing from `lemma.toml`) is a hard fail.

Trust expansion. The Generator may freely add an oracle-shaped axiom by minting a new `[[oracles]]` block (Inline oracle creation). It may not add anything else. To expand the non-oracle trust base, see §5.

5 The proposal queue (asynchronous human approval)

A proposal is the Planner's request for the human to expand the non-oracle trust base. Trust-base expansions of the following kinds require a proposal:

- A new free-floating analytic axiom (e.g., a wider analyticity domain for F^{-1}_{paper}).
- A non-trivial Mathlib import bringing in disputed/heavy infrastructure.
- A statement reformulation (rare; typically forbidden as a weakening of Lemma 5.4).

A proposal is a Markdown file `Proposals/proposal_<NNN>_<slug>.md` naming, in mechanically actionable detail, the Lean axiom to add, the lemma.toml entry to modify, the math content (with paper citation), the numerical win unlocked (with a Scratch/ run cited), and the trust cost.

Async discipline.

1. The Planner files a proposal *and queues a regular sprint in the same invocation* so the loop never stalls. The queued sprint must not depend on the pending proposal.
2. Budget: at most 3 `pending` proposals at any time.
3. The human reviews on their own time, edits the proposal's `Status:` field to `approved` or `rejected`, appends a status-log line, and (if approved) lands the infrastructure (Lean axiom + lemma.toml entry).
4. The Planner reads `Status: approved` on its next invocation and queues a sprint that consumes the new infrastructure.

In our run the proposal queue surfaced one approved proposal (`proposal_001_cr_refined_oracles`: three η-aware oracles $M_\Phi, M_{\Phi'}, M_{\Phi\Phi}$ that unlocked the η^2-tight C_r^{refined} formula) and one pending one (`proposal_003_wider_analyticity_domain`: a paper-citable wider analyticity disk that would lift the Cauchy contour radius by 1.5×).

6 The numerical library and the certified-arb pipeline

`Numerical_Library/` is a Python package indexed by `lemma.toml`'s oracle table. Each oracle module exposes a single `compute_<name>_bound(...)` entry point that returns `OracleResult(bound, ball_value, ...)`, where:

- `ball_value` is a single `float` approximating the sup the oracle bounds;
- `bound` is a `Fraction` rationalized upward from the certified arb interval, with safety parameters `(safety_rel, safety_abs, max_denom)` producing a small but *provably valid* excess over the arb upper bound.

The library wraps the lower-level certified-arb infrastructure at `grothendieck/proof/certified/`, which is the same code base the production proof-graph runner consumes (§7).

Truncation-free oracles. The 2026-05-03 patch (PATCH_NOTES.md) introduced `Numerical_Library/trunc` a parallel oracle suite that evaluates $H_\eta, H'_\eta, H''_\eta$ via Price's-theorem path integrals and chunk-bounds them via real-positive Gaussian majorants—no ϕ-series truncation anywhere in the call graph. The deprecated top-level oracles are kept only to satisfy backwards-compat for older candidates; G7.a forbids any *new* candidate from reaching them.

7 The proof graph (production layer)

The harness above operates on the Lemma 5.4 *candidate workspace* `ProofRefinement/Lemma54/`. Adjacent to it—and connected only through file-based handoff—is the *proof graph*, the production trust boundary at `ProofGraph/`. The proof graph is a static DAG of nodes representing each step in the end-to-end argument $H_\eta \to$ Lemma 5.3 $\to$ Lemma 5.4 $\to$ Theorem 5.1 $\to \mathcal{K}_{G_{\mathrm{lt}}}^{\mathrm{Krivine}}$. Each node has:

- a Lean theorem name (the production-faithful statement),
- an `Effective` companion (instantiated at the locked tuple $\eta = 1/100$, $r = 23/25$),
- a `nodes/<name>.toml` declaring its trust dependencies and registered oracles,
- a runner-side certified-arb computation that emits a `certificate.json` cross-validating every oracle assumption.

The candidate workspace is a *playground* for the harness; the proof graph is the *commitment surface*. A winning candidate is ported to the proof graph by hand: a tightened formula becomes a parallel `Cr_*` definition, a newly-proved analytic axiom drops from the trust base, a tightened numerical bound is reflected in the runner's certificate. This separation is by design: the harness can run autonomously without touching the production trust boundary, and only after a winning candidate has been cross-validated by hand does its content move into `ProofGraph/`.

8 The audit floor and PATCH_NOTES

A subtle but important property of the harness is that retroactive rule-tightening is supported through the `PATCH_NOTES.md` mechanism. When a regression is discovered in the trust base—e.g., the 2026-05-03 finding that every existing oracle inherited a $K_{\max}$-truncated ϕ-series with no rigorous tail bound—the discovery is documented as a binding directive in `PATCH_NOTES.md`: a new gate is added (G7), a parallel truncation-free pipeline is built, a per-candidate audit re-evaluates each V_K under the post-patch oracles, and the resulting ${C_r^{\mathrm{patched}}}_K$ becomes the new floor.

The audit table records, for each V_K, three numbers: ${C_r^{\mathrm{val}}}_K$ (declared), ${C_r^{\mathrm{patched}}}_K$ (production-evaluated under post-patch oracles), and a structural label indicating which C_r formula and oracle constellation V_K uses. The table is the basis for the **audit-floor consistency** gate G7.b: a new candidate V_{N+1}'s audit-block header must name the cached audit JSON and the specific ${C_r^{\mathrm{patched}}}_{N+1}$ it claims, and the Evaluator rejects V_{N+1} if the audit JSON's $C_r^{\mathrm{patched}} > C_r^{\mathrm{val}}$.

The take-away for the planner: a sprint cannot beat C_r^{val} by literal-tightening alone. It must *move the audit floor*, by formula-switching, deeper subdivision, sharper joint oracles, or alternative contours. Re-rationalising an oracle ceiling against a non-rigorous floor is not a permitted direction.

9 Iteration loop and slash commands

A complete sprint, from the human's perspective, is a single slash command:

```
/run-next-sprint
```

The skill spawns the Planner with no arguments; it reads the inputs above and writes `Sprint_Contracts/sprint_`
Then it spawns the Generator with the contract path; the Generator edits Lean and terminates with a labeled status. Then it spawns the Evaluator with the same contract path; the Evaluator

runs G1–G7 and writes a verdict. Total wall-clock: 5–30 minutes depending on tier and Lean compile time.

For autonomous operation:

```
/loop 30m /run-next-sprint
```

The harness self-paces; `scripts/harness_preflight.sh` halts the loop if the same goal fails three sprints in a row.

10 Safety properties

The harness is engineered around five guarantees, each enforced mechanically by a different combination of file conventions and gates.

(P1) The sink theorem statement is pinned. V_0 declares `theorem lemma_5_4_search : <signature>`. Every later candidate must declare a theorem with the same name and statement (`lemma.toml:sink_theorem`) The Evaluator runs `##print axioms` on whatever the candidate names `lemma_5_4_search`; weakening the statement renders the `Cval` comparison meaningless and is treated as cheating by code review.

(P2) History is sacrosanct. Older candidates and Sprint Contracts are immutable. Gate G3 rejects any diff touching them. The Lean `olean` cache is wiped per sprint to avoid stale-binary leakage.

(P3) The trust base is on-paper. `lemma.toml` is the only declaration of which seed axioms and which oracles a candidate may use. Any axiom outside this declaration is a gate G4 failure, regardless of how it was introduced.

(P4) The trust base is re-checked every audit. Every oracle is re-run on every G5; the Python implementation is the certificate. A bug in the implementation is caught by the bound-field-vs-recomputed-bound inequality.

(P5) Retroactive regression recovery is supported. When a previously-trusted construction is found unsound (the $K_{\max}$ truncation), a new PATCH_NOTES rule + new gate + new audit table re-classify every prior candidate. The framework does not "patch in place"; the old cache stays, the new gate fails any future candidate that depends on the retired construction.

11 Per-candidate proof graph visualisation

A complementary artifact, useful both for sanity-checking and for paper-figure inclusion, is a *per-candidate Lean dependency graph*: for each V_K, the cone of `theorem`/`lemma`/`def`/`axiom` declarations transitively reachable from `lemma_5_4_search` in V_K together with the seven shared `Lemma54_Paper*.lean` library files.

`paper_artifacts/proof_graph/build_per_candidate.py` parses each candidate, strips Lean comments to suppress false-positive declarations, builds the directed graph by name-reference, restricts to the cone reachable from the entry point, and renders with Graphviz. Outputs:

- `per_candidate/V<K>_proof_graph.{png,svg,pdf,dot}` — one per candidate.
- `proof_graph_progression.gif` — a 29-frame animation of the cone evolving across $V_0, \ldots, V_{28}$.
- `proof_graph_progression.pdf` — the same frames concatenated.

Node convention: `theorem`/`lemma` are rounded boxes, `def` ellipses, `axiom` octagons; colour encodes the source file, with the candidate's own decls in orange. The entry point's border is bolder than the rest. The figures make the *formula × oracle constellation* structure of the audit table immediately visible: when V_N replaces a triangle product by a joint oracle, two former leaves ($M_\Phi, M_{\Phi'}, \mathrm{inv}_{F_p^z}$) become orphaned and disappear; when V_N adopts the merged-numerator form, the u_{zz} branch collapses entirely.

12 Summary of the run that produced the bound

Across 34 sprints (29 candidates, several of which were research-only or salvage runs), the certified C_r^{val} traced

$$8.2\times10^{14} \;\to\; 1.13\times10^{9} \;\to\; 3.79 \;\to\; 1 \;\to\; 1/268{,}000 \xrightarrow{K_{\max}\text{-patch}} 0.3255 \;\to\; 0.2857.$$

The trace shows three regimes:

1. *Numerical tightening of the paper formula* ($V_0 \to V_8$): rectangle retargeting, R-rationalisation, oracle-precision bumps. Score moves $\sim 10^5\times$.
2. *Structural formula refinements* ($V_9 \to V_{26}$): the auxiliary z-identity, joint sup-bounds replacing triangle products, the merged-numerator form, oracle-resolution scans. Score moves $\sim 10^6\times$ on the $K_{\max}$-era scale.
3. *Truncation-free re-baseline* ($V_{27} \to V_{28}$): post-PATCH_NOTES, the score is now reported on the rigorous floor; the apparent regression to 0.33 is the cost of retiring a non-rigorous oracle and the gain is that every future bound is honest at the 10^{-17} scale.

The current best is ${C_r^{\mathrm{val}}}_{V_{28}} = 2/7$, against an audit floor of ${C_r^{\mathrm{patched}}}_{V_{28}} = 0.2810$ on V_{14}'s $C_r^{\phi,u_{zz},\mathrm{joint}}$ formula at three rounds of corner subdivision. The path forward, per `PATCH_NOTES.md~§3a`, is to move the audit floor itself: a fourth subdivision round, a fourth-order Taylor pad in w, a sharper $M_{\Phi'''}$ Cauchy bound, or the wider analyticity domain of `proposal_003`.

13 Provenance

The agent prompts that define the three roles are checked in at `.claude/agents/refinement-\{planner,generat` and were not modified during the run except for two policy updates: (i) `2026-04-30` relocating oracle creation from the proposal queue to inline-Generator self-serve; (ii) `2026-05-03` adding G7 in response to the $K_{\max}$ regression discovery.

The reading list every agent must consult, in priority order: PATCH_NOTES, the candidate it branches from, `Proof/Lib/Lemma54_Paper.lean`, `lemma.toml`, the relevant section of the Braverman et al. paper, and the most recent `Bulletin.md` priority directive.

References

[1] M. Braverman, K. Makarychev, Y. Makarychev, A. Naor. *The Grothendieck constant is strictly smaller than Krivine's bound*. Forum of Mathematics, Pi 1 (2013), e4.